

03_Module_3_Toil_and_Automation

Module 3 — Toil Elimination & Automation

Duration: 8 Hours

3.1 Toil as a System Problem

Definition

Toil is operational work that is **manual, repetitive, automatable, tactical, and has no enduring value**. The service is no better after the work is done — it's just caught up.

Classification:

Work Type	Example	Toil?
Restarting a crashed service (1st time)	Investigating why it crashed	 No — debugging builds knowledge
Restarting a crashed service (10th time)	Same root cause, same manual fix	 Yes — you haven't fixed the root cause
Writing a runbook	Documenting response procedures	 No — adds enduring value
Following the same runbook weekly	Running the same commands by hand	 Yes — automate the runbook
Building a dashboard	First-time setup	 No — creates visibility
Checking the same dashboard daily	For a year, no issues found	 Yes — the alert should tell you
On-call response (new issue)	Investigating a novel problem	 No — engineering work
On-call response (same page, same fix)	50th time for the same alert	 Yes — fix root cause

The 50% Rule

If > 50% of engineering time is toil, the team cannot improve the system. They can only tread water.

Measurement methodology:

1. **Tag time** — have engineers tag their work for 2 weeks (toil vs engineering)
2. **Quantify** — calculate percentage, identify top toil items
3. **Surface** — report to management with cost impact
4. **Prioritise** — allocate sprint time to eliminate the highest-frequency items
5. **Track** — report hours saved per quarter

Toil budget approach:

Treat toil like an error budget. Allocate a maximum percentage of team time to toil. When it exceeds the budget, stop non-essential work and eliminate toil sources.

3.2 Automation Decision Framework

Not all toil should be automated. Use this decision tree:

Is it toil? (manual, repetitive, automatable, tactical)

- └─ No → Leave it (it's engineering work)
- └─ Yes →
 - How often does it happen?
 - └─ Rarely (< 1/month) → Document it (runbook), don't automate
 - └─ Occasionally (weekly) → Script it (Python/Bash)
 - └─ Frequently (daily) → Build a tool/service
 - └─ Constantly (every minute) → Design it out of the system

Does the automation add risk?

- └─ Yes → Add guardrails (approval gates, dry-run mode)
- └─ No → Automate fully

Can we measure the impact?

- └─ Yes → Track hours saved, report ROI
 - └─ No → Add measurement first, then automate
-

3.3 Automation Patterns for SRE

Pattern 1: Self-Healing

Instead of alerting a human and waiting for a response, the system detects and fixes the problem automatically.

Example — Database connection pool:

```
# Self-healing: detect and restart
import psycpg2
import subprocess
import time

def check_and_heal():
    try:
        conn = psycpg2.connect(
            host="db-primary",
            port=5432,
            connect_timeout=3
        )
        conn.close()
        print("Database is healthy")
        return
    except Exception as e:
        print(f"Database unhealthy: {e}")
        print("Attempting restart...")
        subprocess.run([
            "systemctl", "restart", "postgresql"
        ], check=True)
        time.sleep(10)
        # Verify
        check_and_heal()
```

```
if __name__ == "__main__":
    check_and_heal()
```

Self-healing maturity levels:

Level	Behaviour	Example
L0	Nothing	Alert fires, human responds
L1	Detect + alert	System knows it's broken, pages human
L2	Detect + auto-remediate	System fixes itself, notifies human
L3	Detect + remediate + verify	Fixes itself, checks it worked, escalates if not
L4	Predictive	Predicts failure, prevents it before it happens

Pattern 2: ChatOps

Move operational tasks into chat, making them visible, auditable, and accessible to the whole team.

Example commands:

```
!deploy transfer-service v2.1.4 production
!restart atm-api lagos-01
!status atm-network
!incident open SEV-2 "Mobile transfers failing"
!runbook show ATM_HIGH_ERROR_RATE
```

Pattern 3: GitOps

Use Git as the single source of truth for operations:

Git repository structure:

```
ops/
├── infrastructure/
│   ├── terraform/
│   │   ├── production/
│   │   └── staging/
│   └── kubernetes/
│       ├── deployments/
│       ├── services/
│       └── configmaps/
├── monitoring/
│   ├── prometheus/
│   │   ├── rules/
│   │   └── alerts/
│   └── grafana/
│       └── dashboards/
├── runbooks/
│   ├── atm-high-error-rate.md
│   └── db-connection-exhaustion.md
└── policies/
    ├── error-budget-policy.yml
    └── deployment-policy.yml
```

GitOps workflow: 1. Change is proposed via PR 2. PR is reviewed by peers 3. CI runs validation (lint, syntax check, dry-run) 4. PR is merged to main 5. CD applies the change automatically 6. Drift detection reverts

unauthorised changes

Pattern 4: Policy-as-Code

Enforce operational policies through code, not documentation.

Example — Open Policy Agent (OPA):

```
# deployment_policy.rego
# Only allow deployments if error budget is healthy

package deployment

default allow = false

allow {
    input.action == "deploy"
    error_budget_remaining >= 0.2 # At least 20% remaining
}

allow {
    input.action == "rollback" # Rollbacks are always allowed
}

error_budget_remaining := data.error_budget.remaining_ratio
```

3.4 Automation Tools at Scale

Python — The SRE Swiss Army Knife

Python is used for: health checks, log parsing, API automation, data processing, incident tooling.

Production-grade health check:

```
import requests
import json
import sys
import logging
from datetime import datetime

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s"
)

def health_check(endpoint: str, timeout: int = 5) -> dict:
    result = {
        "endpoint": endpoint,
        "timestamp": datetime.utcnow().isoformat(),
        "status": "unknown",
        "latency_ms": 0,
        "error": None
    }
    try:
        start = datetime.utcnow()
        resp = requests.get(endpoint, timeout=timeout)
        latency = (datetime.utcnow() - start).total_seconds() * 1000
        result["latency_ms"] = round(latency, 2)
```

```

    if resp.status_code == 200:
        result["status"] = "UP"
    elif resp.status_code < 500:
        result["status"] = "DEGRADED"
    else:
        result["status"] = "DOWN"
        result["error"] = f"HTTP {resp.status_code}"
except requests.Timeout:
    result["status"] = "DOWN"
    result["error"] = "timeout"
except requests.ConnectionError:
    result["status"] = "DOWN"
    result["error"] = "connection refused"

return result

# Run checks
services = [
    ("https://mobile.unionbank.com/health", "Mobile Banking"),
    ("https://atm.unionbank.com/status", "ATM Network"),
    ("https://transfer.unionbank.com/health", "Transfer Service"),
]

for url, name in services:
    result = health_check(url)
    level = logging.ERROR if result["status"] == "DOWN" else logging.INFO
    logging.log(level, f"{name}: {result}")

```

Ansible — Configuration Management at Scale

Ansible ensures all servers are configured consistently.

Playbook structure:

```

# site.yml – top-level playbook

- name: Apply base configuration to all servers
  hosts: all
  become: yes
  roles:
    - common          # users, SSH, NTP, logging
    - monitoring       # node_exporter, telegraf

- name: Apply database configuration
  hosts: databases
  become: yes
  roles:
    - postgresql
    - backup

- name: Apply application configuration
  hosts: applications
  become: yes
  roles:
    - app-deploy
    - health-check

```

Terraform — Infrastructure as Code

Multi-environment structure:

```
# environments/production/main.tf
module "atm_network" {
  source = "../../modules/atm-network"

  environment = "production"
  region      = "eu-west-2"
  instance_count = 4
  instance_type = "t3.large"

  tags = {
    Environment = "production"
    Service     = "atm-network"
    CostCenter  = "CC100"
    Owner       = "sre-team"
  }
}

module "monitoring" {
  source = "../../modules/monitoring"

  environment      = "production"
  prometheus_retention = "30d"
  grafana_admin     = true
}
```

3.5 Measuring Automation Impact

Before	After	Savings
Manual DB restart: 15 min/day, 5x/week	Auto-heal: 0 min	60 hrs/year
Manual log check: 30 min/day	ELK search: 30 sec	120 hrs/year
Manual deployment: 2 hrs/deploy, 2x/week	CI/CD: 5 min/deploy	180 hrs/year
Manual capacity check: 1 hr/month	Prometheus alert: instant	12 hrs/year
Total		372 hrs/year per engineer

3.6 Classroom Activity: Automation Blueprint

Time: 1 hour

Instructions:

Each group picks one toil task from their current work and produces an automation blueprint:

1. **Describe the task** — what happens, frequency, time cost
2. **Quantify the toil** — hours per week, cost to the business
3. **Choose the pattern** — self-healing? ChatOps? GitOps? Policy-as-code?
4. **Architecture** — draw the flow (system diagram)
5. **Guardrails** — how do you prevent automation from making things worse?
6. **ROI** — hours saved, implementation cost, break-even time

Present and vote — which automation would save the most time across Union Bank?

Key Terms

Term	Definition
Toil	Manual, repetitive, automatable work with no lasting value
Self-healing	System detects and fixes problems automatically
ChatOps	Running operations through chat interfaces
GitOps	Git as the single source of truth for operations
Policy-as-Code	Enforcing operational policies through code
ROI	Return on Investment — time saved vs implementation cost
50% Rule	Maximum toil budget for an SRE team
Drift	Configuration changes that deviate from the desired state

Further Reading

- [Google SRE Book, Chapter 5 — Eliminating Toil](#)
- [GitOps Principles](#)
- [Open Policy Agent Documentation](#)
- [The Art of Automation — Google SRE](#)